posco

설비 고장을 예측하는 AI 데이터 분석

예외처리 및 종합 실습

데이터 처리 파이프라인

posco

CONTENTS

데이터 처리 파이프라인

PART 1
파이프라인 설계
종합 실습 개요 · 파이프라인 설계

PART 2
읽기와 분류
데이터 읽기 · 조건 분류

PART 3
통계와 방어
통계 함수 · 불량 데이터 방어

PART 4
리포트 작성
리포트 구성 · 파일 저장

종합 실습 개요 — 무엇을 만드는가

배운 문법을 모아 하나의 분석 프로그램을 완성 — 새 문법은 없음
앞 단계 결과가 뒤 단계의 재료가 되는 연결 구조 = 파이프라인

변수 · 조건문 · 반복문 · 함수 · 파일 입출력 · 예외처리

ICT 검사 CSV를 읽어 부품별로 정리하고, 불량을 걸러내고, 통계를 내어 리포트로 저장

만들 프로그램

데이터 분석 실무의 가장 기본적인 흐름이다.

새 문법은 없다

한 단계씩 쌓아 올리므로 막혀도 어느 단계인지 추적 가능

파이프라인 단계별 흐름

읽기 → 분류 → 계산 → 방어 → 저장. 각 단계는 앞 단계 결과를 이어받는다.



CSV 19행 → rows 19 → 부품 5종(미분류 0) → 숫자 16건 → 유효 15건 → 리포트

줄어드는 지점마다 이유를 말할 수 있어야 파이프라인을 이해한 것

PRACTICE

실습 1. CSV 읽기

> 목표
>
> **CSV의 헤더와 데이터 행을 분리하는 read_csv_data(path) 함수 완성**

단계

① 입력 파일: data/09_ict_inspection_dirty.csv (DATA_PATH) 지정

② csv.reader로 읽고 next(reader)로 첫 줄(헤더) 분리

③ (header, rows) 순서로 반환 — 모두 리스트 형태

④ 파일이 없으면 FileNotFoundError 처리 후 ([], []) 반환

⑤ RUN_TO_STEP=1 실행 후 헤더·데이터 행 수 확인

> **실습 조건 |** 입력 data/09_ict_inspection_dirty.csv · 반환 (header, rows) · 없는 파일 ([], [])

예상 결과

완료 기준: 헤더 7항목 · 데이터 19행 · 파일 없을 때 ([], [])

PRACTICE

실습 2. 부품별로 분류하기

목표

실습 1의 rows를 부품명(row[1]) 기준으로 딕셔너리에 묶기

단계

① 실습 1의 rows 19행을 for문으로 반복

② 두 번째 열 row[1]을 부품명으로 사용

③ 부품명을 키(key)로 갖는 by_part 딕셔너리 준비

④ 각 부품의 목록에 해당 CSV 행(row)을 그대로 추가

⑤ 부품명 빈칸·열 부족은 dropped에 세고 제외

⑥ RUN_TO_STEP=2 → 부품 종류와 미분류 행 수 확인

실습 조건 | 입력 실습 1의 rows 19행 · 부품명 row[1] · 반환 (by_part, dropped)

예상 결과

완료 기준: A2·B2·D2·E2·F2 총 5종 / 부품명 누락 0행

딕셔너리에 값을 모으는 두 가지 방법

조건문으로 준비하는 방법과 setdefault로 한 줄에 처리하는 방법

CODE

```
if p not in by_part:
    by_part[p] = []
by_part[p].append(row)

# 같은 동작을 한 줄로
by_part.setdefault(p, []).append(row)
```

EXPLAIN

if p not in by_part

조건문 방식은 흐름이 눈에 보임

setdefault

setdefault는 키가 없을 때만 기본값을 넣고 그 값을 돌려줌

두 방식의 결과는 완전히 동일
조건을 빠뜨리면 `KeyError` 발생

CHECKPOINT

같은 데이터로 두 방식을 각각 돌려 결과가 같은지 직접 비교

PRACTICE

실습 3. 통계 함수 만들기

> 목표
>
> **측정값 열의 개수·평균·최댓값·최솟값을 계산하는 함수**

단계

① calc_stats(rows, column=2) 함수 만들기

② 각 row[column]을 float로 변환해 values에 추가

③ ValueError·IndexError인 행만 건너뛰기

④ 유효 숫자가 없으면 None 반환 (0으로 나누기 방지)

⑤ 숫자가 있으면 count·mean·max·min 계산

⑥ 딕셔너리로 반환하고 RUN_TO_STEP=3으로 확인

> **실습 조건 |** 입력 rows, column=2 · float 변환만 적용 · -999는 아직 포함

예상 결과

완료 기준: 숫자 16건 · 평균 1716.64 · 최소 -999 (아직 범위 검증 전)

PRACTICE

실습 4. 불량 데이터 걸러내기

목표

검증 기준 0~5000으로 잘못된 측정값을 제외하고 기록

단계

① filter_valid_rows(rows, lo=0, hi=5000) 함수 작성

② row[2]를 float로 변환 — 문자·빈칸이면 제외

③ 측정값 열이 부족하면 IndexError로 제외

④ 범위 밖(예: -999)은 raise ValueError로 제외

⑤ rejected에 (CSV 줄 번호, 검사ID, 제외 사유) 기록

⑥ 정상 행은 valid에 모아 (valid, rejected) 반환

실습 조건 | 입력 rows, lo=0, hi=5000 · 제외: 문자/빈칸/열 부족/범위 밖 · 반환 (valid, rejected)

예상 결과

완료 기준: 유효 15건 / 제외 4건 (CSV 17~20행, 검사ID 9001~9004)

좋은 리포트의 원칙

핵심 먼저, 단위 붙이기, 구분선 활용 — 세 가지 원칙으로 정리한다.

01

핵심 먼저

전체 · 유효 · 제외 건수를
맨 위에 놓는다.

읽는 사람이 가장 먼저
알고 싶은 정보부터.

02

단위 붙이기

건수에는 '건', 측정값에는
항목명과 단위 확인을.

그래야 사람이 바로
의미를 이해할 수 있다.

03

구분선 활용

줄을 나누고 **구분선**으로
영역을 가른다.

리포트 줄들을 **리스트에 모아**
한 번에 파일로 쓴다.

데이터를 다 분석해도 사람이 못 읽으면 소용없음 — 읽는 사람이 가장 알고 싶은 수치부터 배치

PRACTICE

실습 5. 리포트 파일로 저장하기

목표

검증된 데이터를 요약한 09_report.txt 파일 만들기

단계

① save_report(path, total_rows, valid_rows, rejected, stats) 작성

② 전체 19행·유효 15건·제외 4건을 lines에 기록

③ 평균 1897.68 · 최대 4366.23 · 최소 0.0 추가

④ 측정값의 물리 단위는 미기재 — 임의 단위 붙이지 않기

⑤ lines를 "\n".join()으로 묶어 UTF-8 텍스트 저장

⑥ RUN_TO_STEP=5 실행 후 09_report.txt 직접 열어보기

실습 조건 | 입력 valid·rejected·stats · 출력 09_report.txt · 측정값 단위 임의 표기 금지

예상 결과

완료 기준: 09_report.txt 생성 / 요약 19·15·4 / 평균 1897.68

저장된 리포트 파일 확인

실제로 만들어진 09_report.txt의 내용

CHECK

파일을 열어
직접 눈으로 확인

숫자만 있고 단위가 없으면 다시 물어보게 됨
제외된 행이 몇 건인지 함께 남기는 것이 핵심

분석 결과는 남겨야 근거가 됨
화면 출력은 프로그램이 끝나면 사라짐

09_report.txt

```
=== ICT 검사 분석 리포트 ===
전체 19행 · 유효 15건 · 제외 4건

DIVIDER

측정값 통계

평균 – 1897.68
최대 – 4366.23
최소 – 0.0
```

PRACTICE

실습 6. 부품별 통계 검증하기

목표

유효 데이터만 다시 묶어 부품별 개수 합계를 검증하기

단계

① 원본 rows가 아닌 실습 4의 valid 15행을 사용

② group_by_part(valid)로 정제된 행을 다시 분류

③ 부품마다 calc_stats(해당 부품 행, 2)의 count 합산

④ 합계(check_sum)와 전체 len(valid)를 비교

⑤ RUN_TO_STEP=6 → 부품별 개수와 True 확인

실습 조건 | 입력 정제한 valid 15행 · 다시 부품별 분류 · 부품별 합계와 len(valid) 비교

예상 결과

완료 기준: A2·B2·D2·E2·F2 각 3건 / 합계 15 = 유효 15 → True